

1. I would make the Row Key for this dataset [2-digit hash prefix]#[DOL_VEHICLE_ID] because this key does not begin with a sequential or highly concentrated attribute, so it will prevent many reads and writes from landing in the same tablet and creating a hotspot. The hash prefix allows inserts and scans to distribute more evenly across nodes, which allows vehicles to be efficiently queried by City or Make.
2. Column Families: id (vehicle identifiers), geo (location related attributes), vehicle (vehicle attributes like make and model), ev_specifics (EV-specific technical or program fields). This helps with distributed read performance because it allows fewer bytes per read and better compression and locality since the information is separated into well related categories such that BigTable does not need to read unrelated information and can easily get the related information from specific columns.
3. Hashed Shard Key has balanced cluster performance due to effective hotspot prevention, but query routing is weaker and less direct for analytical filters. Range-Based Shard Key has very efficient query routing, but it has hotspot risk which leads to less balanced cluster performance.

Part 4:

Average latencies:

Fast - 205.46 ms

Safe - 189.40 ms

A real-world scenario where I would intentionally choose the faster primary-only write over the risk of data loss is page view events like total page views or number of clicks. This type of scenario would benefit from fast speed and high throughput, and losing a small amount of data would be fine because the data is not critical. If you had to log every time a user clicks, it would be better to prioritize speed so that you can capture as many user clicks as possible, and losing a few clicks wouldn't matter in the big picture.